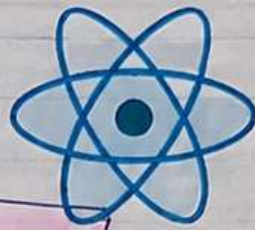


React



"Small Steps
Big Career"
😊

Interview Questions

50 Questions Every Frontend Developer Should Know

Learn
Practice
Build
Grow ❤️

1 What is React?

- React is a JavaScript library for building user interfaces.
- Developed by Facebook (now Meta).
- It allows us to build reusable UI components and efficiently update the UI when data changes.
- React follows a component-based architecture and uses a virtual DOM to optimize rendering.

Key Points:

- ☑ Developed by: Facebook (2013)
- ☑ Maintained by: Meta
- ☑ Type: JavaScript library (for building UIs)
- ☑ Architecture: Component-based
- ☑ Uses: Virtual DOM
- ☑ Language: JavaScript (with JSX)

"React is not just a library, it's an ecosystem."

2 Why use React?

- Component reusability
- Faster rendering with Virtual DOM
- Strong community support
- Large ecosystem (React Router, Redux, etc.)
- Can be used for web and mobile (React Native)
- Backed by Meta and widely adopted in the industry

Popular companies using React:

- Meta (Facebook)
- Instagram
- Netflix
- Airbnb
- Uber
- and many more...

3 React vs Framework

Aspect	React (Library)	Framework (e.g. Next.js)
Definition	A UI library for building components	A complete solution with built-in tools and structure
Control	Gives more flexibility	More opinionated
Routing	Not built-in (use React Router)	Built-in routing
Features	Focuses on UI	Includes SSR, routing, API routes, optimization etc.
Example	React	Next.js (built on React)



Interview Tip:

If asked "Is React a framework?", clearly explain the difference with examples.

"Understand the concepts, not just the syntax." 😊

JSX in React

Write HTML in JavaScript



"Code
Learn
Repeat
Grow"

"JSX makes it easier to write and understand React components."

1 What is JSX?

- JSX (JavaScript XML) is a syntax extension for JavaScript.
- It looks like HTML but is actually converted to `React.createElement()` by Babel before it runs in the browser.
- It allows us to write UI structure in a more readable and declarative way.

Example: JSX vs `React.createElement`

```
// JSX
const element = (
  <h1>Hello React! </h1>
);

// Transpiled to
const element = React.createElement(
  'h1',
  null,
  'Hello React!'
);
```

2 Rules of JSX

1. Return a single parent element.
2. Use camelCase for attributes.
3. Close all tags (even self-closing).
4. Use `{}` to embed JavaScript expressions.
5. `class` → `className` (since `class` is a reserved word).
6. `for` → `htmlFor`.

Why not HTML directly?

- Browsers cannot understand JSX directly.
- JSX is converted to plain JavaScript using Babel.
- It helps in writing UI and logic together.



3 Embedding Expressions

- We can use JavaScript expressions inside `{}`.
- We can use variables, function calls, ternary operators etc.

```
const name = "Abhishek";
const element = <h2>Hello, {name}! </h2>;
const num = 5;
const result = <p>2 + 3 = {2 + 3} </p>;
```

We can also use:

- Function calls → `{getName()}`
- Conditional (ternary) → `{isLoggedIn ? 'Logout' : 'Login'}`
- Array mapping → `{items.map(...)}`

4 JSX vs HTML

Feature	JSX (React)	HTML
Syntax	HTML-like + JavaScript	Pure HTML
Attributes	camelCase (<code>className</code> , <code>htmlFor</code>)	kebab-case (<code>class</code> , <code>for</code>)
JavaScript support	Yes (inside <code>{}</code>)	No
Self-closing tags	Required (<code></code>)	Optional
Compiled	Yes (by Babel)	No (native)

Interview Tip:

- JSX is not a string or HTML.
- It improves developer experience.
- Know how it gets transpiled.

"JSX is syntactic sugar, but it makes React development much more elegant."



React Components

Building Blocks of React

"Everything in React is a component."

"Components
Make
Complex UIs
Simple" 😊

1 What is a Component?

- A component is a reusable piece of UI in React.
- It can be a function or a class that returns JSX.
- Components help in breaking the UI into smaller, manageable parts.
- Each component has its own logic, UI and can accept inputs (props).

Example: Functional Component:

```
function Welcome() {  
  return (  
    <div>  
      <h2>Hello, React! </h2>  
    </div>  
  );  
}
```

Returns
JSX

2 Types of Components

1. Functional Components (Recommended)
 - Written using JavaScript functions.
 - Use hooks (useState, useEffect, etc).
 - Simpler and cleaner.
2. Class Components (Older approach)
 - Written using ES6 classes.
 - Use lifecycle methods (e.g. componentDidMount).
 - Still used in some older codebases.

Example: Class Component.

```
class Welcome extends React.Component {  
  render() {  
    return (  
      <div>  
        <h2>Hello, React! </h2>  
      </div>  
    );  
  }  
}
```

3 Component Composition

- Components can be nested inside other components.
- Helps in reusability and better structure.
- We can pass data from parent to child using props.

Example: Nested Components

```
function App() {  
  return (  
    <div>  
      <Navbar />  
      <Welcome />  
    </div>  
  );  
}
```

Child
Components

4 Benefits of Components

- ☑ Reusability
- ☑ Better code organization
- ☑ Easier maintenance
- ☑ Improves collaboration
- ☑ Helps in building complex UIs



Interview Tip:

Talk about real-world examples where you used components to build reusable UI (e.g. Button, Modal, Card).

"Good component design makes a big application feel like a set of small problems." 😊

5 Naming Convention

- Component names should start with a capital letter (PascalCase).
- Example: MyComponent, UserProfile, Navbar ✓
- Avoid: mycomponent, userprofile ✗

"Small Components
→ Big Possibilities" ♥

Props vs State in React

Understanding Data Flow

"Right Data Right Component Brighter UI"

"Props are how you configure a component, state is how it remembers."



1 What are Props?

- Props (short for properties) are inputs passed to a component from a parent component.
- They are read-only (immutable).
- Used to pass data, functions, or even components.
- Helps in making components reusable.

Example: Props

```
function Greeting({ name }) {
  return <h2>Hello, {name}! </h2>;
}

function App() {
  return <Greeting name="Abhishek"/>;
}
```

↑ Passing data through props

2 What is State?

- State is data managed inside a component.
- It can change over time (mutable).
- Used to handle dynamic data (e.g. user input, API response, UI changes).
- Updating state triggers a re-render.

Example: State

```
import { useState } from "react";

function Counter() {
  const [count, setCount] = useState(0);
  return (
    <div>
      <p>Count: {count} </p>
      <button onClick={() => setCount(count+1)}>
        Increment
      </button>
    </div>
  );
}
```

3 Props vs State (Comparison)

Feature	Props	State
Definition	Input to a component	Data managed inside a component
Mutability	Immutable (read-only)	Mutable (can change)
Set by	Parent component	Component itself (using useState)
Purpose	Pass data / configuration	Handle dynamic data
Re-render	When parent re-renders (or prop value changes)	When state is updated
Example	<Component name="Abhi" />	const [count, setCount] = useState(0);



Interview Tip:

- Props make components reusable.
- State makes components interactive.
- Use props for static/external data.
- Use state for data that changes over time.

4 Can we modify props?

- No, props are read-only.
- If you need to change a value, use state.
- Mutating props can lead to unexpected behavior and bugs.

Incorrect ✗

```
function User({ name }) {
  name = "New Name"; // ✗ Cannot reassign
  return <h2>{name} </h2>;
}
```

Correct Approach ✓

Use state if you need to change it.

5 When to use what?

- Use Props → When data comes from a parent or is static.
- Use State → When data changes based on user actions or other events.
- Often both are used together in real-world applications.

"Props start the conversation, State keeps it going."



useState in React

Managing State in Functional Components

"Practice
Build
Improve
Repeat"
😊

"useState lets you add state to functional components."

1 What is useState ?

- useState is a React Hook that allows you to add state in functional components.
- It returns an array with two values:
 1. current state value
 2. function to update the state.
- State is preserved between re-renders.

Example: Basic Counter

```
import { useState } from "react";  
function Counter() {  
  const [count, setCount] = useState(0);  
  return (  
    <div>  
      <p>Count: {count} </p>  
      <button onClick={() => setCount(count+1)}>  
        Increment  
      </button>  
    </div>  
  );  
}
```

Initial state value

2 Syntax

```
const [state, setState] = useState(initialValue);
```

- state → current value
- setState → function to update value
- initialValue → initial state (can be any type)



Interview Tip:

- The name can be anything (e.g. count, setCount) using destructuring.
- Initial value can be a number, string, object, array, etc.

3 Examples with Different Data Types

Number

```
const [count, setCount] = useState(0);
```

String

```
const [name, setName] = useState("");
```

Boolean

```
const [isLoggedIn, setIsLoggedIn] =  
  useState(false);
```

Array

```
const [items, setItems] = useState([]);
```

Object

```
const [user, setUser] = useState({  
  name: "Abhishek",  
  age: 25  
});
```

When updating objects/arrays,
always create a new copy
(don't mutate directly).

4 Updating State

- Use the setter function (setState) to update state.
- State updates are asynchronous.
- React may batch multiple state updates for better performance.
- If new state depends on previous state, use functional update.

Example: Functional Update

```
setCount(prev => prev + 1);
```

Use this when new state
depends on previous state.

5 Key Points

- ☑ State updates trigger a re-render.
- ☑ State is local to the component.
- ☑ useState can be used multiple times in a single component.
- ☑ Do not mutate state directly (especially objects and arrays).

"State changes
UI."
😊

useEffect in React

Handling Side Effects in Functional Components

"Effects
Connect
React to
the real World."
😊

"useEffect lets you perform side effects in your components."

1 What is useEffect ?

- useEffect is a React Hook that lets you perform side effects in functional components.
- Side effects include API calls, subscriptions, DOM manipulation, timers, etc.
- It runs after the component has been rendered (committed to the screen).

Syntax

```
useEffect(() => {  
  // side effect code  
  
  return () => {  
    // cleanup code (optional)  
  };  
}, [dependencies]);
```

← effect function

← optional cleanup function

← dependency array

2 When does useEffect run ?

- By default, it runs after every render.
- You can control when it runs using dependency array.
- It runs asynchronously after the browser paints the UI.

Key Points:

- ☑ useEffect runs after render.
- ☑ It doesn't block the UI.
- ☑ Can return a cleanup function.
- ☑ Dependency array controls execution.
- ☑ Used for side effects (not for rendering).

3 Examples

1. No dependency array (runs after every render)

```
useEffect(() => {  
  console.log("Component rendered");  
});
```

2. Empty dependency array (runs only once)

```
useEffect(() => {  
  console.log("Runs only on mount");  
}, []);
```

Useful for
API calls on
component mount

3. With dependency array (runs when value changes)

```
useEffect(() => {  
  console.log("Count changed:", count);  
}, [count]);
```

Runs only when
'count' changes

4. Cleanup function (e.g. event listener)

```
useEffect(() => {  
  const handleScroll = () => {  
    console.log("Scrolling...");  
  };  
  window.addEventListener("scroll", handleScroll);  
  
  return () => {  
    window.removeEventListener("scroll", handleScroll);  
  };  
}, []);
```

Cleanup runs
on unmount

4 Common Use Cases

- ☑ Fetching data from an API
- ☑ Subscribing to events (WebSocket, DOM events)
- ☑ Timers (setTimeout, setInterval)
- ☑ Manually changing the DOM
- ☑ Cleaning up resources (event listeners, subscriptions).

5 Common Mistakes

- ✗ Using useEffect for rendering logic.
- ✗ Forgetting the dependency array.
- ✗ Not cleaning up (memory leaks).
- ✗ Adding unnecessary dependencies.

★ Interview Tip:

Explain when useEffect runs, how dependency array works, and why cleanup is important. Also mention common use cases with real examples.

"UseEffect is powerful, but use it wisely." 😊

Other Important Hooks in React

Small Hooks, Big Impact

"Hooks Simplify Complex Logic." 😊

"Hooks let you use React features in functional components."

1 useRef()

- useRef returns a mutable reference object whose .current value persists across renders.
- It does not trigger re-render when updated.
- Commonly used to access DOM elements or store mutable values.



Interview Tip:

- Use useRef for DOM access, timers, previous values, etc.
- It persists across renders.
- Updating .current does not re-render.

Example: Accessing a DOM element

```
import { useRef } from "react";
function Input() {
  const inputRef = useRef(null);
  const handleFocus = () => {
    inputRef.current.focus();
  };
  return (
    <div>
      <input ref={inputRef} />
      <button onClick={handleFocus}>Focus</button>
    </div>
  );
}
```

Directly access DOM

2 useMemo()

- useMemo is used to memoize expensive computations.
- It only recalculates the value when dependencies change.
- Helps in performance optimization.

Example: Expensive Calculation

```
import { useMemo } from "react";
function Square({ num }) {
  const squared = useMemo(() => {
    console.log("Calculating...");
    return num * num;
  }, [num]);
  return <div>Square: {squared}</div>;
}
```

Only re-runs when 'num' changes.

3 useCallback()

- useCallback is used to memoize functions.
- Returns the same function reference until dependencies change.
- Useful when passing functions to child components to avoid unnecessary re-renders.

Example: Memoized Function

```
import { useCallback } from "react";
function Parent() {
  const handleClick = useCallback(() => {
    console.log("Button clicked");
  }, []);
  return <Child onClick={handleClick} />;
}
```

Same function reference across renders (until deps change).

4 useContext()

- useContext lets you consume context values without prop drilling.
- Useful for global data like theme, auth, etc.

Example: Using Context

```
import { useContext } from "react";
const ThemeContext = React.createContext();
function Button() {
  const theme = useContext(ThemeContext);
  return <button className={theme}>Click</button>;
}
```

Access global data directly.

5 useReducer()

- useReducer is an alternative to useState for complex state logic.
- It's useful when state has multiple sub-values or when next state depends on previous state.

Example: Counter with useReducer

```
function reducer(state, action) {
  switch(action.type) {
    case 'inc': return { count: state.count + 1 };
    case 'dec': return { count: state.count - 1 };
    default: return state;
  }
}
const [state, dispatch] = useReducer(reducer, { count: 0 });
```

Better for complex state logic (e.g. multiple actions).

"Choose the right hook for the right problem."



Custom Hooks in React

Reusing Logic, The React Way

"React Hooks = Cleaner Code Happier You" 😊

"Custom Hooks let you extract and reuse component logic."

1 What are Custom Hooks ?

- Custom Hooks are JavaScript functions that start with "use" and can call other hooks.
- They help you reuse stateful logic across multiple components.
- They make your components cleaner, smaller and more maintainable.

Example: useCounter (Custom Hook)

```
import { useState } from "react";

function useCounter(initialValue = 0) {
  const [count, setCount] = useState(initialValue);
  const increment = () => setCount(count + 1);
  const decrement = () => setCount(count - 1);
  return { count, increment, decrement };
}
```

Reusable logic in a hook!

2 Why use Custom Hooks ?

- Avoids code duplication.
- Separates logic from UI.
- Makes components easier to read and test.
- Encourages reusable and modular code.
- Useful for common tasks like API calls, form handling, local storage, etc.

Key Points:

- ☑ Custom hooks start with "use".
- ☑ Can use other hooks inside.
- ☑ Help in code reusability.
- ☑ Keep components clean.
- ☑ Follow the same rules as built-in hooks (e.g. only call at top level).
- ☑ Can return any value (state, functions, etc.).



3 Example: useFetch (API Call Hook)

- A custom hook to fetch data from an API.
- Handles loading, error and data states.
- Can be reused across different components.

```
import { useState, useEffect } from "react";
function useFetch(url) {
  const [data, setData] = useState(null);
  const [loading, setLoading] = useState(true);
  const [error, setError] = useState(null);
  useEffect(() => {
    fetch(url)
      .then(res => res.json())
      .then(data => setData(data))
      .catch(err => setError(err))
      .finally(() => setLoading(false));
  }, [url]);
  return { data, loading, error };
}
```

Using the Custom Hook in a Component

```
import React from "react";
import useFetch from "./useFetch";

function Users() {
  const { data, loading, error } = useFetch(
    "https://jsonplaceholder.typicode.com/users"
  );
  if (loading) return <p>Loading...</p>;
  if (error) return <p>Error: {error.message}</p>;
  return (
    <ul>
      {data.map(user => (
        <li key={user.id}>{user.name}</li>
      ))}
    </ul>
  );
}
```

4 Common Use Cases

- ☑ Fetching data from APIs
- ☑ Handling form input and validation
- ☑ Managing localStorage / sessionStorage
- ☑ Listening to window events (resize, scroll)
- ☑ Debouncing / throttling values

"Custom Hooks turn repeated logic into reusable power." 😊



Building a Custom Hook (Step by Step)

Let's Build: useLocalStorage

"Abstract Today, Reuse Forever."

"A practical example to understand how custom hooks do work."



1 Problem

- We often need to store data in localStorage (e.g. theme, user preferences).
- It involves repetitive code.
- Let's create a custom hook to handle this in a reusable way.



What we want ?

- ☑ Store a value in localStorage
- ☑ Persist it across page reloads
- ☑ Provide a state-like interface [value, setValue]
- ☑ Handle JSON parsing/stringifying
- ☑ Work for any data type

2 Create the Hook

useLocalStorage.js

```
import { useState } from "react";

function useLocalStorage(key, initialValue) {
  // 1. Get value from localStorage (if exists)
  const [value, setValue] = useState(() => {
    const storedValue = localStorage.getItem(key);
    return storedValue ? JSON.parse(storedValue) : initialValue;
  });

  // 2. Update localStorage when value changes
  const updateValue = (newValue) => {
    setValue(newValue);
    localStorage.setItem(key, JSON.stringify(newValue));
  };

  return [value, updateValue];
}

export default useLocalStorage;
```

3 Using the Hook

Example Component

```
import React from "react";
import useLocalStorage from "../useLocalStorage";

function ThemeToggle() {
  const [theme, setTheme] = useLocalStorage(
    "theme", "light"
  );

  const toggleTheme = () => {
    setTheme(theme === "light" ? "dark" : "light");
  };

  return (
    <div>
      <p>Current Theme: {theme}</p>
      <button onClick={toggleTheme}>
        Toggle Theme
      </button>
    </div>
  );
}

export default ThemeToggle;
```

4 How it Works ?

1. useState initializes with value from localStorage (or initialValue).
2. When value changes, we update both state and localStorage.
3. The hook returns [value, updateValue] just like useState.
4. Now you can reuse this hook anywhere in your app.

Output (UI)

Current Theme: dark

Toggle Theme

The value persists even after page reload!

5 More Examples

- useWindowSize() → track window width & height
- useOnlineStatus() → check if user is online/offline
- useDebounce() → debounce a value
- usePrevious() → get previous value of a state
- useToggle() → toggle between true/false



Key Takeaways:

- ☑ Custom hooks help in reusability.
- ☑ They can manage complex logic.
- ☑ Follow the same rules as React hooks.
- ☑ Keep them small, clean and focused.
- ☑ Use them to solve real problems.

"Write hooks that work for you, not against you."



More Useful Hooks in React

Small Hooks, Big Possibilities

"Practice Hooks Build Better Apps."



"React provides many hooks to solve real-world problems easily."

1 useReducer()

- Alternative to useState for complex state logic.
- Useful when state has multiple sub-values or next state depends on previous state.
- Works with a reducer function (similar to Redux).

Example: Counter with useReducer

```
import { useReducer } from "react";
function reducer(state, action) {
  switch(action.type) {
    case "inc": return { count: state.count + 1 };
    case "dec": return { count: state.count - 1 };
    default: return state;
  }
}
const [state, dispatch] = useReducer(reducer, { count: 0 });
<button onClick={() => dispatch({ type: "inc" })}>+</button>
<button onClick={() => dispatch({ type: "dec" })}>-</button>
```

Great for todo apps, forms, etc.

3 useDeferredValue()

- Defers updating a value until more urgent updates (like typing) are done.
- Useful for improving performance in search, filtering, or large lists.

Example: Deferred Search

```
import { useState, useDeferredValue } from "react";
function Search() {
  const [query, setQuery] = useState("");
  const deferredQuery = useDeferredValue(query);
  return (
    <div>
      <input value={query} onChange={(e) => setQuery(e.target.value)}
        placeholder="Search..." />
      <p>Searching for: {deferredQuery}</p>
    </div>
  );
}
```

Input stays smooth even with large data.

2 useTransition()

- Lets you handle non-urgent updates without blocking the UI.
- Useful for heavy computations or filtering large lists.
- Returns a tuple: [isPending, startTransition]

Example: Search with useTransition

```
import { useState, useTransition } from "react";
function Search() {
  const [query, setQuery] = useState("");
  const [isPending, startTransition] = useTransition();
  const handleChange = (e) => {
    const value = e.target.value;
    startTransition(() => {
      setQuery(value);
    });
  };
  return (
    <div>
      <input onChange={handleChange}
        placeholder="Search..." />
      {isPending && <p>Loading...</p>}
      <p>Search: {query}</p>
    </div>
  );
}
```

Keeps UI responsive during heavy tasks!

4 useId()

- Generates a unique and stable ID for accessibility.
- Useful when working with labels, forms, and ARIA attributes.

Example: Accessible Input

```
import { useId } from "react";
function InputField() {
  const id = useId();
  return (
    <div>
      <label htmlFor={id}>Name:</label>
      <input id={id} type="text" />
    </div>
  );
}
```

Helps in accessibility and avoids ID conflicts.

5 useImperativeHandle()

- Customizes the instance value that is exposed when using ref with forwardRef.
- Useful when you want to expose specific methods to parent components.



Key Takeaways:

- ☑ React provides many hooks for different use cases.
- ☑ Choose the right hook based on the problem.
- ☑ Hooks make components more reusable and maintainable.
- ☑ Keep your code clean and focused.

6 useLayoutEffect()

- Runs synchronously after DOM mutations but before the browser paints.
- Useful for reading layout from the DOM (e.g. element size, position).
- Use only when necessary (can block rendering).

"Hooks are not just features, they are a better way to think about React."



Even More Useful Hooks in React

Solve Real Problems, Write Better Code

"Different problems, different hooks — same powerful React."

"Consistency with Hooks builds confidence."



1 useRef()

- Let's you persist a mutable value without causing re-renders.
- Useful for accessing DOM elements, storing timers, previous values, etc.

Example: Accessing an input element

```
import { useRef } from "react";
function FocusInput() {
  const inputRef = useRef(null);
  const handleFocus = () => {
    inputRef.current.focus();
  };
  return (
    <div>
      <input ref={inputRef} type="text" />
      <button onClick={handleFocus}>Focus </button>
    </div>
  );
}
```

Directly access a DOM element!

3 useCallback()

- Memoizes a function reference.
- Useful when passing functions to child components to prevent unnecessary re-renders.
- Similar to useMemo, but for functions.

Example: Passing callback to a child

```
import { useState, useCallback } from "react";
function Parent() {
  const [count, setCount] = useState(0);
  const handleClick = useCallback(() => {
    setCount(count + 1);
  }, [count]);
  return <Child onClick={handleClick} />;
}
```

Function reference remains same until "count" changes!

5 useDebugValue()

- Used for custom hooks.
- Displays a label in React DevTools.
- Helpful for debugging custom hooks.

Example: Custom hook with debug label

```
import { useState, useDebugValue } from "react";
function useOnlineStatus() {
  const [isOnline, setIsOnline] = useState(navigator.onLine);
  useDebugValue(isOnline ? "Online" : "Offline");
  return isOnline;
}
```

Shows value in React DevTools!

2 useMemo()

- Memoizes the result of a computation.
- Re-runs only when dependencies change.
- Useful for expensive calculations (e.g. filtering large lists).

Example: Expensive calculation

```
import { useMemo, useState } from "react";
function ProductList({ items }) {
  const [search, setSearch] = useState("");
  const filteredItems = useMemo(() => {
    console.log("Filtering...");
    return items.filter(item =>
      item.name.toLowerCase().includes(search.toLowerCase())
    );
  }, [items, search]);
  return (
    <div>
      <input value={search} onChange={e => setSearch(e.target.value)}
        placeholder="Search..." />
      <ul>
        {filteredItems.map(item => (
          <li key={item.id}>{item.name} </li>
        ))}
      </ul>
    </div>
  );
}
```

Avoids unnecessary re-computation

4 useContext()

- Consumes values from React Context.
- Avoids prop drilling.
- Useful for theme, authentication, global state, etc.

Example: Using Theme Context

```
import { useContext } from "react";
import { ThemeContext } from "../ThemeContext";
function Navbar() {
  const theme = useContext(ThemeContext);
  return (
    <nav style={{ background: theme === "dark" ? "#333" : "#fff" }}>
      My App
    </nav>
  );
}
```

Access global data directly!

★ Key Takeaways:

- ☑ Each hook solves a specific problem.
- ☑ Use the right hook for better performance.
- ☑ Keep dependencies accurate.
- ☑ Hooks make components more reusable, clean and maintainable.
- ☑ Explore and combine hooks to build powerful features.

"Master the small hooks, and you can build big things!"



Real World Examples + Best Practices

Putting Hooks Together

"Combine multiple hooks to build something useful."

"Hooks
Make React
More Powerful
and Flexible."



1 Example: Search with Debounce

- useState for search text
- useEffect with setTimeout (debounce)
- useDebounceValue (custom hook)
- Fetch API after delay

SearchComponent.jsx

```
import { useState, useEffect } from "react";
import { useDebounceValue } from "../useDebounceValue";

function SearchComponent() {
  const [query, setQuery] = useState("");
  const debouncedQuery = useDebounceValue(query, 500);

  useEffect(() => {
    if (debouncedQuery) {
      console.log("Fetching results for:", debouncedQuery);
      // API call here
    }
  }, [debouncedQuery]);

  return (
    <input
      value={query}
      onChange={(e) => setQuery(e.target.value)}
      placeholder="Search..."
    />
  );
}
```

Combines
useState +
useEffect +
custom hook!

2 Example: Theme Switcher (Context + Custom Hook)

- useContext to access theme
- Custom hook to toggle theme
- Global state without prop drilling

ThemeContext.jsx

```
import { createContext, useContext, useState } from "react";

const ThemeContext = createContext();

export function ThemeProvider({ children }) {
  const [theme, setTheme] = useState("light");
  const toggleTheme = () => {
    setTheme(theme === "light" ? "dark" : "light");
  };

  return (
    <ThemeContext.Provider value={{ theme, toggleTheme }}>
      {children}
    </ThemeContext.Provider>
  );
}

export function useTheme() {
  return useContext(ThemeContext);
}
```

Use context
for global state
like theme,
auth, etc.

3 Example: Form Handling with Custom Hook

- Use useState for form values
- Custom hook to handle input changes
- Reusable for multiple forms

useForm.js

```
import { useState } from "react";

export function useForm(initialValues) {
  const [values, setValues] = useState(initialValues);
  const handleChange = (e) => {
    const { name, value } = e.target;
    setValues({ ...values, [name]: value });
  };

  return { values, handleChange, setValues };
}
```

Reusable
hook for
forms!

4 Example: Window Size Tracker

- useState for width/height
- useEffect to listen to resize event
- Clean up event listener

useWindowSize.js

```
import { useState, useEffect } from "react";

export function useWindowSize() {
  const [size, setSize] = useState({
    width: window.innerWidth,
    height: window.innerHeight,
  });

  useEffect(() => {
    const handleResize = () => {
      setSize({ width: window.innerWidth, height: window.innerHeight });
    };

    window.addEventListener("resize", handleResize);
    return () => window.removeEventListener("resize", handleResize);
  }, []);

  return size;
}
```

Don't forget
to clean up
event listeners
in useEffect!

★ Best Practices

- ☑ Use hooks at the top level (not inside loops/conditions).
- ☑ Keep hooks focused (single responsibility).
- ☑ Use descriptive names for custom hooks (useXyz).
- ☑ Always clean up side effects (event listeners, timers, etc.).
- ☑ Avoid unnecessary re-renders (useMemo, useCallback).
- ☑ Reuse hooks to keep your code DRY.

"The real power of React comes
when you combine hooks to solve
real problems in simple, reusable ways."

Keep Building



Creating Production Ready Custom Hooks

From Simple to Powerful

"Good hooks are small, focused, reusable and well-tested."

"Clean Hooks
Lead to
Clean Apps."



1 useToggle()

- Manages a boolean state.
- Useful for modals, sidebars, dropdowns.
- Simple but commonly used.

useToggle.js

```
import { useState, useCallback } from "react";
export function useToggle(initialValue = false) {
  const [value, setValue] = useState(initialValue);
  const toggle = useCallback(() => setValue(v => !v), []);
  const setTrue = useCallback(() => setValue(true), []);
  const setFalse = useCallback(() => setValue(false), []);
  return { value, toggle, setTrue, setFalse }
}
```

Usage:

```
import { useToggle } from "../useToggle";
function Modal() {
  const { value: open, toggle, setFalse } = useToggle();
  return (
    <>
      <button onClick={toggle}>Open Modal</button>
      {open && (
        <div className="modal">
          <p>Modal Content</p>
          <button onClick={setFalse}>
        </div>
      )}
    </>
  )
}
```

Same hook,
many use cases:
modals, menus,
dropdowns!

2 usePrevious()

- Stores previous value of a state or prop.
- Useful for comparing values.
- Helps in animations, form dirty check, etc.

usePrevious.js

```
import { useRef, useEffect } from "react";
export function usePrevious(value) {
  const ref = useRef();
  useEffect(() => {
    ref.current = value;
  }, [value]);
  return ref.current
}
```

Usage:

```
function Counter() {
  const [count, setCount] = useState(0);
  const prevCount = usePrevious(count);
  return (
    <div>
      <p>Current: {count}</p>
      <p>Previous: {prevCount}</p>
      <button onClick={() => setCount(count + 1)}>
        Increment
      </button>
    </div>
  );
}
```

Very useful
for tracking
changes!

3 useOnlineStatus()

- Tracks if user is online or offline.
- Listens to browser events.
- Useful for showing offline/online banner.

useOnlineStatus.js

```
import { useState, useEffect } from "react";
export function useOnlineStatus() {
  const [isOnline, setIsOnline] = useState(navigator.onLine);
  useEffect(() => {
    const handleOnline = () => setIsOnline(true);
    const handleOffline = () => setIsOnline(false);
    window.addEventListener("online", handleOnline);
    window.addEventListener("offline", handleOffline);
    return () => {
      window.removeEventListener("online", handleOnline);
      window.removeEventListener("offline", handleOffline);
    };
  }, []);
  return isOnline;
}
```

Usage:

```
import { useOnlineStatus } from "../useOnlineStatus";
function Status() {
  const isOnline = useOnlineStatus();
  return (
    <div>
      {isOnline ? (
        <p style={{ color: "green" }}>
          ☒ You are Online
        </p>
      ) : (
        <p style={{ color: "red" }}>
          ☐ You are Offline
        </p>
      )}
    </div>
  );
}
```

Small hook,
big impact!

★ Best Practices

- ✓ Keep hooks small and focused.
- ✓ Use descriptive names.
- ✓ Return simple and consistent values.
- ✓ Handle cleanup (event listeners, timers, subscriptions).
- ✓ Use useCallback / useMemo when needed.
- ✓ Write proper TypeScript types (for larger projects).
- ✓ Test your hooks.
- ✓ Document with examples.

"Custom hooks are
building blocks. The better
you design them, the easier
it is to scale your app."



Small Hooks
Bigger Possibilities!



Advanced Patterns with Custom Hooks

Take It to the Next Level

"Combine hooks, handle async logic, and solve real-world problems."

"Hooks today,
better apps
tomorrow."



1 useFetch() – Data Fetching Hook

- Handles API calls (loading, error, data).
- Reusable across components.
- Keeps your components clean.

useFetch.js

```
import { useState, useEffect } from "react";
export function useFetch(url) {
  const [data, setData] = useState(null);
  const [loading, setLoading] = useState(true);
  const [error, setError] = useState(null);

  useEffect(() => {
    const fetchData = async () => {
      try {
        const res = await fetch(url);
        const json = await res.json();
        setData(json);
      } catch (err) {
        setError(err.message);
      } finally {
        setLoading(false);
      }
    };
    fetchData();
  }, [url]);
  return { data, loading, error };
}
```

Handles
loading,
error and
data neatly!

Usage:

```
import { useFetch } from "../useFetch";

function Users() {
  const { data, loading, error } = useFetch(
    "https://jsonplaceholder.typicode.com/users"
  );
  if (loading) return <p>Loading...</p>;
  if (error) return <p>Error: {error}</p>;
  return (
    <ul>
      {data.map(user => (
        <li key={user.id}>{user.name}</li>
      ))}
    </ul>
  );
}
```

★ Key Takeaways:

- ✓ Custom hooks can handle complex logic.
- ✓ Separate concerns (data, loading, error).
- ✓ Makes components clean and readable.
- ✓ Reuse the same hook across your app.
- ✓ Error handling is important in real apps.

2 useLocalStorage() with Error Handling

- Improved version with error handling.
- Handles SSR (window undefined).
- Useful for production apps.

useLocalStorage.js

```
import { useState, useEffect } from "react";
export function useLocalStorage(key, initialValue) {
  const [value, setValue] = useState(initialValue);
  useEffect(() => {
    try {
      const item = window.localStorage.getItem(key);
      if (item) setValue(JSON.parse(item));
    } catch (err) {
      console.error("Error reading localStorage", err);
    }
  }, [key]);
  const setStoredValue = (newValue) => {
    try {
      setValue(newValue);
      window.localStorage.setItem(key, JSON.stringify(newValue));
    } catch (err) {
      console.error("Error setting localStorage", err);
    }
  };
  return [value, setStoredValue];
}
```

More robust
version for
real-world
use!

3 Combining Multiple Hooks

- You can use multiple custom hooks together.
- Build powerful features with small hooks.
- Example: useOnlineStatus + useFetch

Example: User List (only when online)

```
import { useOnlineStatus } from "../useOnlineStatus";
import { useFetch } from "../useFetch";

function UserList() {
  const isOnline = useOnlineStatus();
  const { data, loading, error } = useFetch(
    "https://jsonplaceholder.typicode.com/users"
  );
  if (!isOnline) return <p>You are offline.</p>;
  if (loading) return <p>Loading users...</p>;
  if (error) return <p>Error: {error}</p>;
  return (
    <ul>
      {data.map(user => (
        <li key={user.id}>{user.name}</li>
      ))}
    </ul>
  );
}
```

Small hooks
can be combined
to create
powerful
features!

4 Tips for Production

- Keep hooks focused (single responsibility).
- Handle errors and edge cases.
- Use meaningful names and TypeScript for larger projects.
- Test your hooks (with Jest + React Testing Library).
- Document what the hook does, inputs and outputs.

"Custom hooks are not just a feature,
they are a mindset — write once,
reuse everywhere."



Code Modular. Code Better.



Wrap Up: Key Takeaways & Next Steps

Small Hooks Today, Bigger Apps Tomorrow

"It's not just about using hooks, it's about using them the right way."

"Learn
Build
Share
Grow"



1 What We Learned

- ✓ React Hooks simplify state and lifecycle management.
- ✓ Built-in hooks like `useState`, `useEffect`, `useContext` solve common problems.
- ✓ Custom hooks help in writing reusable and modular code.
- ✓ Advanced hooks (`useReducer`, `useCallback`, etc.) improve performance and scalability.
- ✓ Hooks can be combined to build real-world features.
- ✓ Clean, focused, and well-tested hooks make better apps.

2 Why Hooks Matter

- Write less boilerplate code.
- Make components more reusable.
- Handle complex logic in a cleaner way.
- Improve code readability and maintainability.
- Build modern, scalable, and high-performance applications.
- Essential skill for any modern React developer.

"Hooks give you the power to do more with less code."



3 Best Practices Recap

- ✓ Use hooks only at the top level.
- ✓ Follow the Rules of Hooks.
- ✓ Use meaningful names for custom hooks.
- ✓ Keep hooks focused (single responsibility).
- ✓ Handle loading, error, and edge cases.
- ✓ Avoid unnecessary re-renders (`useMemo`, `useCallback`).
- ✓ Write clean, readable and well-documented code.
- ✓ Test your hooks with real use cases.

4 Common Mistakes to Avoid

- ✓ Calling hooks inside loops, conditions or nested functions.
- ✓ Overusing `useEffect`.
- ✓ Forgetting dependencies in `useEffect`, `useCallback`.
- ✓ Creating large, unstructured custom hooks.
- ✓ Not handling cleanup (event listeners, timers, subscriptions).
- ✓ Re-computing heavy values instead of using `useMemo`.
- ✓ Using state when a ref would be better (`useRef`).
- ✓ Ignoring error handling in async hooks.

5 Next Steps

- Practice by building small projects.
- Try combining multiple hooks.
- Explore popular open-source hooks (e.g. React Query, SWR, Zustand with hooks).
- Read and understand real-world codebases.
- Build your own custom hook library.
- Keep learning and stay updated with the React ecosystem.
- Apply these concepts in real projects (portfolio/work).

6 Practice Ideas

- Build a to-do app (`useState` + `useEffect`).
- Create a theme switcher (`useContext`).
- Make a searchable list with debounce (`useDebounce`).
- Build a window size tracker (`useWindowSize`).
- Create a form with validation (custom hook).
- Build a data fetching hook with error handling.
- Combine multiple hooks in a small dashboard app.

Practice
Build
Experiment
Improve!



Final Takeaways:

- ✓ Hooks are a game-changer in modern React.
- ✓ They help you write cleaner, modular, and scalable code.
- ✓ Start small, practice consistently.
- ✓ Combine hooks to solve bigger problems.
- ✓ Always think about performance, readability, and reusability.
- ✓ The more you practice, the more confident you become.
- ✓ React is evolving — keep learning!

"Small, reusable hooks can lead to big, maintainable and powerful applications."

Keep Practicing
Keep Building
See You in the Next Notes!



Same
Learner
Bigger Builder!

